Support Centre

Implementation prompt for another Replit project

Based on the Support Centre implemented in E-Migration Assist. Upload this document to the destination project and ask its agent to implement the prompt below.

The current feature is a support request form, not live chat. It saves requests locally, emails the support team, and forwards requests to the Eride Support Hub, which issues the canonical ticket reference. The source app's Customer Support admin page is a placeholder, not a working inbox.

Implementation instructions

Implement a fully functional Support Centre matching the appearance and behaviour below. Build the complete submission flow, not just the interface.

Use this project's existing framework, database, email service, routing, and component conventions. Do not replace its architecture or change unrelated features.

Adapt the product name, support inbox, and Support Hub product identity to this project. Do not send this project's requests under E-Migration Assist's product identity.

1. What this feature does

Any visitor, including someone who is not logged in, can open a floating panel, choose a category, optionally enter a name, provide an email and message, submit, receive on-screen confirmation with the Hub reference when available, and submit another request.

Behind the scenes, validate and apply spam protection; save locally; send an internal notification email; forward server-side to the Hub; then save and return the Hub reference.

This is not a chatbot, live-chat system, or customer conversation inbox.

2. Floating launcher

Mount one shared widget in the top-level application layout across routes, including logged-out pages. The source also displays it on admin pages.

- Fixed bottom-right, approximately 20px from each edge.
- Rounded pill button with dark navy background #0a1628.
- White "Support Centre" text and teal outlined headphones icon.
- Thin teal outline, soft dark shadow, and visible keyboard focus ring.
- Teal accent: hsl(187 38% 52%). Hover background: #0d1d36.
- Above ordinary content; source z-index is 60.
- Desktop: icon and text. Below the small-screen breakpoint: icon only, accessible name "Open Support Centre".
- Hide the launcher while the panel is open.

3. Support panel

Open without navigation or a full-screen backdrop.

Desktop: bottom-right with 20px margins, width 380px, maximum height 80vh, approximately 16px corner radius, navy background, subtle teal border, soft shadow, scrollable body and header that stays visible within the panel.

Mobile: full viewport width, anchored bottom-right without outer margin, maximum height 90vh, and scrollable body so every field and button is reachable.

Header: headphones icon inside a subtle teal circle; title "Support Centre"; subtitle "We usually reply by email"; close button on the right.

Use #0d1d36 for the header and #0a1628 for the body. Provide a labelled dialog, labelled form fields, accessible open/close buttons and visible focus indicators. Escape closes the panel. The source has no modal focus trap or background-blocking overlay; do not turn it into a full-screen modal.

4. Form fields and copy

Show a softly teal-tinted welcome box:

> Welcome to [PRODUCT NAME]. Submit a support query, report a technical issue, ask a question, or request assistance with your application.

Adapt the last phrase if this product is not application-related.

Category label: "How can we help?" Default: support_query.

- support_query -> Support query
- technical_issue -> Technical issue / bug
- payment_account -> Payment / account
- general_question -> General question

Name: label "Name (optional)"; placeholder "Your name"; optional; server maximum 120 characters.

Email: label "Email"; placeholder "you@example.com"; required; email input with format validation; server maximum 200 characters.

Message: label "Message"; placeholder "Tell us how we can help..."; required; initially four textarea rows; trim before submission; server length 1-4,000 characters.

Name and Email appear side by side on desktop and stack on mobile. Inputs have dark backgrounds, subtle light borders, white text, muted placeholders and teal focus styling.

Include a hidden website honeypot: offscreen, excluded from tab order, autocomplete disabled, hidden from assistive technology and normal users.

Submit button: "Send request". While submitting: "Sending..." with spinner. Disable and guard against repeated submission while pending.

5. Widget state behaviour

Maintain open/closed, category, name, email, message, honeypot, submitting, success, ticket reference and error states.

- Closing an unfinished form preserves its draft while the widget remains mounted.
- Closing after success resets the widget for its next opening.
- "Submit Another Request" resets without closing.
- Reset restores the default category and clears fields, errors, success and reference.
- Drafts are not stored in browser storage in the source.
- A failed submission preserves entered information for correction or retry.

Client validation messages:

> Please enter your email so we can reply.
>
> Please enter a short message so we can help.

Display API errors in the panel. The general retry fallback is:

> Could not send your request. Please try again.

6. Submission endpoint

Implement a public endpoint equivalent to POST /api/support, Content-Type: application/json.

```
{
  "category": "support_query",
  "name": "Optional name or null",
  "email": "visitor@example.com",
  "message": "The support request",
  "pagePath": "/current-page",
  "website": ""
}
```

Use this project's API URL/base-path helper. Never hardcode localhost or another project's backend. Capture window.location.pathname as pagePath, without query strings or hashes that may contain sensitive information.

Validate independently on the server:

- Category: one of the four allowed values.
- Message: trimmed, required, 1-4,000 characters.
- Name: trimmed, optional/nullable, maximum 120.
- Email: trimmed, required, valid format, maximum 200.
- Page path: trimmed, optional/nullable, maximum 300.
- Website: optional string.

Invalid input: HTTP 400 with {"error":"Invalid input","details":[]} populated with validation issues.

Check the honeypot first. A website value containing non-whitespace text returns HTTP 201 {"ok":true}, without saving, emailing or forwarding.

Apply a per-IP limit of 20 requests per hour. Exceeding it returns HTTP 429, a Retry-After header, and:

```
{"error": "Too many requests. Please try again later."}
```

The source limiter is process-local/in-memory. If this project runs multiple instances, use an existing shared limiter if available and explain that difference.

7. Local database persistence

Create an equivalent support_requests table using the project's existing database tools:

- id: generated UUID primary key.
- category: required text.
- message: required text.
- name: nullable text.
- email: text; required by the submission API.
- page_path: nullable text.
- status: required text, default "new".
- hub_ticket_reference: nullable text.
- hub_synced_at: nullable timestamp with timezone.
- created_at: timestamp with timezone, default now.

Save locally before email or Hub forwarding. If persistence fails, return HTTP 500 {"error":"Failed to record request"}. Do not display success if it was not saved.

Support requests are separate from leads, applications, referrals and sales enquiries. Do not create those records as a side effect.

8. Internal notification email

After saving locally, send an internal notification to SUPPORT_NOTIFY_EMAIL. Configure the new project's actual inbox, not E-Migration Assist's address.

Reuse its existing server-side sender. The source supports Resend and configured production SMTP; equivalent delivery behaviour matters more than copying the infrastructure.

Subject: [Support] <category label>

Body: product/app name, category, name or "(not provided)", email, page path or "(unknown)", local request UUID, and full message. For the technical_issue email category label, the source uses "Technical issue".

Set Reply-To to the submitter's email. Use a verified sender address for From, never the visitor's email.

Email delivery is non-blocking: do not turn a saved request into an error if sending fails. Log failures with the local request ID and keep credentials server-side.

Do not add a visitor acknowledgement email. The source sends only the internal team notification.

9. Eride Support Hub integration

Implement a server-side adapter. The source targets https://eride-support-hub.replit.app. Make the URL configurable using SUPPORT_HUB_URL, and support SUPPORT_HUB_PRODUCT_ID.

Verify the current Hub contract before integration. Do not assume permission to register or modify products.

Product identity

Use this project's own Hub identity. Resolve from:

1. Explicit SUPPORT_HUB_PRODUCT_ID override.

2. Cached prior successful lookup.

3. GET /api/support/products, matching this project's product code.

The source matches EMA and has an EMA-specific fallback UUID. Do not copy either into an unrelated product. If identity cannot be established, ask for the correct non-secret product identifier rather than silently filing requests under EMA.

Cache successful lookups for the process lifetime. Do not cache failed lookups.

Category mapping

- support_query -> general_support
- general_question -> general_support
- technical_issue -> technical_bug
- payment_account -> payment_issue

Hub payload

POST /api/support/tickets server-side with Content-Type: application/json. Adapt product-specific wording:

```
{
  "productId": "<resolved product ID>",
  "category": "<mapped category>",
  "issueSummary": "<normalized message, first 140 characters>",
  "whatWereYouTryingToDo":
    "Contacting support via the [PRODUCT NAME] Support Centre widget.",
  "whatWentWrong": "<full trimmed message>",
  "pageOrStep": "<page pathname or null>",
  "stepsToReproduce": null,
  "applicationReference": null,
  "reporterName": "<trimmed name or Support Centre user>",
  "reporterType": "applicant",
  "reporterEmail": "<submitted email>",
  "reporterWhatsapp": null,
  "deviceType": null,
  "browser": null,
  "canContact": true,
  "consent": true,
  "deviceInfo": {
    "ua": "",
    "os": "",
    "viewport": "",
    "language": "",
    "referrerPath": "",
    "hrefPath": "<page pathname or empty string>"
  }
}
```

Normalize the summary by collapsing whitespace to single spaces, trimming and taking the first 140 characters. If empty, use "Support Centre request".

The source uses reporterType "applicant" and sets canContact/consent true without a separate checkbox. Confirm those meanings fit the destination product and current Hub requirements; do not interpret them as marketing consent.

The source intake is unauthenticated. Verify whether that remains true; any required credentials stay server-side. Use a 10-second timeout per external request.

Reference handling

Read ticketReference from the Hub response; accept ticketNumber as a compatibility fallback. The Hub owns numbering. Do not generate an EMA-style sequence locally or hardcode a screenshot's reference.

On success, save hub_ticket_reference, set hub_synced_at and return the actual reference.

Hub failure

If lookup, forwarding or reference retrieval fails, retain the local request, do not undo submission, log the issue with its local ID, and return success with ticketReference null.

Do not invent a reference or claim successful Hub delivery. The source has no retry/reconciliation worker; do not claim guaranteed delivery.

10. API success response

After persistence and the attempted Hub forwarding, return HTTP 201:

```
{
  "ok": true,
  "id": "<local request UUID>",
  "ticketReference": "<Hub reference or null>"
}
```

Await the Hub attempt so its reference can appear in the confirmation. Do not await the email notification.

11. Success screen

Replace the form inside the same panel with a large teal outlined check-circle icon and centred content:

> Thank you!
>
> Your support request has been submitted successfully.

When a reference exists:

> Your ticket reference is [ACTUAL REFERENCE].

Then:

> Our support team will review your request and get back to you via email as soon as possible.

Button: "Submit Another Request".

Use muted light body text, a bold teal reference, and teal button with dark text. If the reference is absent, omit only that sentence. Never display "null", a fabricated reference, or an empty placeholder.

12. Analytics

If Meta Pixel already exists, emit Contact after successful submission with only:

```
{
  "content_name": "Support Widget",
  "content_category": "support"
}
```

Never send name, email, message or ticket reference to analytics. Do not install Meta Pixel solely for this feature.

13. Scope boundaries

Do not add live/AI chat, attachments, phone/WhatsApp fields, customer ticket tracking, conversation threads, assignment/priority/status workflows, automatic customer acknowledgement emails, lead/application creation, or an in-app admin inbox presented as if already implemented.

The source Customer Support admin page is only a protected placeholder. Actual triage is separate through the shared Hub and support email.

14. Verification and handover

Verify all of the following:

1. Launcher and panel render correctly on desktop and mobile.
2. Open, close, Escape, draft preservation and reset work.
3. Logged-out visitors can submit.
4. Required fields, invalid email and server length limits are enforced.
5. Duplicate clicks are blocked while submitting.
6. Valid submission creates a local database record.
7. Team notification has the correct destination and Reply-To.
8. Hub forwarding uses this project's product, not EMA.
9. The actual Hub reference is saved and displayed.
10. Hub/email failures do not discard a saved request.
11. Database failure shows an error, not success.
12. Honeypot submissions trigger no database, email or Hub actions.
13. Rate limiting returns HTTP 429 with Retry-After.
14. Secrets remain server-side; analytics includes no personal data.
15. Existing features continue to work.

Mock external services for automated tests. Do not create real tickets or send test emails to live recipients without approval.

At handover, report what was implemented, remaining configuration, what was verified, and any unverified integration. Do not describe a configured-but-untested email or Hub connection as confirmed working.